

Résumé Exécutif : Ce document fournit une analyse technique exhaustive des développements réalisés sur l'application de Gestion d'Interventions Techniques. Il détaille l'architecture globale découplée (RESTful API), la structure de données, les algorithmes métier, l'intégration frontend SPA/PWA responsive, le moteur de formulaires dynamiques, ainsi que les couches d'interaction matériel (WebRTC Caméra Live & Géolocalisation GPS).

1. ARCHITECTURE GLOBALE & STACK TECHNOLOGIQUE

L'application est construite sur une architecture modernisée découplée et modulaire, garantissant une séparation nette entre la couche de données (Backend Laravel REST API) et la couche de présentation (Frontend SPA Mobile & Dashboard Web).

Domaine / Couche	Technologie Principale	Rôle & Dépendances Techniques
Backend Core	PHP 8.2+ / Laravel 11	Gestion du framework MVC, routage RESTful, ORM Eloquent, injection de dépendances, services métier et événements.
Sécurité & Sécurisation API	Laravel Sanctum & Spatie RBAC	Authentification par tokens Bearer révocables ('personal_access_tokens'), contrôle d'accès basé sur les rôles (Admin, Technicien, Commercial, Client).
Base de Données	MySQL 8.0	Stockage relationnel structuré avec contraintes d'intégrité référentielle, clés étrangères et indexations d'optimisation.
Frontend Mobile	Single Page App (Blade / JS ES6+)	Interface réactive et responsive SPA ('/mobile'), Tailwind CSS dark mode customisé, FontAwesome 6 Pro, client HTTP Fetch asynchrone.
Interaction Matériel (Hardware)	WebRTC & HTML5 APIs	API Caméra Live ('navigator.mediaDevices.getUserMedia'), capture instantanée sur Canvas HTML5, Géolocalisation GPS ('navigator.geolocation'), Signature tactile Canvas.
Génération Documents PDF	Barryvdh DomPDF	Moteur de rendu PDF côté serveur pour les rapports d'intervention techniques et guides d'audit.

2. STRUCTURE DES FICHIERS ET UTILITÉ TECHNIQUE CHRONOLOGIQUE

A. Couche Base de Données & Migrations (Persistence Engine)

Fichier Migration / Model	Utilité Technique & Modifications Apportées
database/migrations/2026_07_18_100001_extend_formulaires_dynamiques_tables.php	Étend la structure des formulaires dynamiques : contrainte d'unicité 'type_intervention_id', ajout des colonnes 'placeholder', 'valeur_par_defaut', 'condition_affichage' (JSON) sur la table 'questions', et libellé d'affichage sur 'choix_questions'. Étend l'enum MySQL 'type_reponse'.
database/migrations/2026_08_12_195450_create_notifications_table.php	Création de la table système 'notifications' de Laravel pour stocker les notifications persistantes envoyées aux techniciens et administrateurs.
database/migrations/2026_08_13_000001_add_logical_fields_to_rapports_table.php	Enrichissement de la table 'rapports' avec des champs terrains réalistes : 'travaux_effectues' (Text), 'observations' (Text), 'recommandations' (Text), 'statut_equipement' (Enum: Conforme, Partiellement conforme, Non conforme, Remplacé), et 'qrcode_scanne' (String).
database/migrations/2026_08_13_000002_extend_questions_type_reponse_enum.php	Extension de l'enum 'type_reponse' pour inclure les types 'Video' et 'Materiaux', permettant des formulaires dynamiques à réponse vidéo et matériels.
database/seeder/FormulaireSeeder.php	Seeder complet créant des formulaires dynamiques ultra-réalistes pour les 8 types d'intervention (154 questions professionnelles au total : Fibre, WiFi, Caméra, Maintenance, Réparation, Routeur, Switch, GPS).

B. Couche Controllers & API Endpoints (Backend API)

Controller Backend	Fichier Source	Fonctionnalités & Utilité Technique
BaseApiController	app/Http/Controllers/Api/BaseApiController.php	Classe parente fournissant un format de réponse JSON unifié (`successResponse`, `errorResponse`) avec codes HTTP standardisés (200, 201, 400, 401, 403, 404, 422).
AuthController	app/Http/Controllers/Api/AuthController.php	Connexion utilisateur (`POST /api/login`), génération et retour du token Sanctum Bearer, déconnexion (`POST /api/logout`) et récupération du profil authentifié (`GET /api/me`).
InterventionController	app/Http/Controllers/Api/InterventionController.php	Gestion du cycle de vie des interventions : liste filtrée (`index`), détails avec relations (`show`), acceptation (`accept`), démarrage avec vérification mode de présence (`start`), et validation finale (`validate`). Isolation des droits techniciens.
FormulaireController	app/Http/Controllers/Api/FormulaireController.php	Récupération des questions dynamiques associées au type d'intervention (`showByIntervention`) et soumission sécurisée des réponses (`storeResponse`), restreinte exclusivement aux techniciens.
RapportController	app/Http/Controllers/Api/RapportController.php	Création et mise à jour du rapport technique d'intervention (`storeOrUpdate`), traitement de l'upload multi-fichiers (photos `photos/rapports`, vidéos `videos/rapports`), gestion du scan QR code et génération/téléchargement PDF (`downloadPdf`).
ProfileController	app/Http/Controllers/Api/ProfileController.php	Gestion du profil technicien : lecture des statistiques et infos (`show`), mise à jour des coordonnées (`update`), upload d'avatar (`uploadPhoto`), et modification sécurisée du mot de passe (`updatePassword`).
NotificationController	app/Http/Controllers/Api/NotificationController.php	Gestion des notifications : consultation paginée (`index`), notifications non lues (`unread`), marquage à lu individuel (`markAsRead`) et groupé (`markAllAsRead`).
TrackingController	app/Http/Controllers/Api/TrackingController.php	Initialisation de session de suivi GPS (`start`), enregistrement périodique des coordonnées GPS (`addPoint`), et fermeture de session (`stop`).

C. Couche Services & Événements (Business Logic Layer)

Classe Service / Event	Fichier Source	Comportement et Logique Métier
InterventionService	app/Services/InterventionService.php	Machine à états centralisée de l'intervention. Gère les transitions de statut (Planifiée ? Acceptée ? En cours ? Formulaire rempli ? Terminée) et déclenche l'envoi automatique des notifications correspondantes.
RapportService	app/Services/RapportService.php	Service de création, mise à jour et suppression des rapports d'intervention. Contrôle des autorisations d'accès client (uniquement si validé).
Classes Notifications	app/Notifications/*	7 notifications personnalisées (`InterventionAssigneeNotification`, `InterventionAccepteeNotification`, etc.) générant des alertes en base et en temps réel.

D. Couche Frontend Mobile SPA & Interface Utilisateur

Composant Frontend	Emplacement Fichier	Fonctionnalité & Rôle UX/UI
Application SPA Mobile	resources/views/mobile/app.blade.php	Application Web Monopage (SPA) autonome intégrant 12 modules réactifs : Login, Dashboard avec KPIs, Liste Missions, Détails avec onglets, Formulaire dynamique, Rapport, Notifications, Profil, Modaux Caméra/QR/Start/Config.
Client API JavaScript	app.blade.php (Section Script)	Wrapper `apiFetch(endpoint, method, body, isFormData)` gérant automatiquement le token Bearer Sanctum, les en-têtes JSON/Multipart, la détection 401 et les messages d'erreur.

Composant Frontend	Emplacement Fichier	Fonctionnalité & Rôle UX/UI
API Caméra Live WebRTC	app.blade.php (#modal-live-camera)	Accès caméra direct en temps réel (`getUserMedia`), viseur visuel avec overlay, commutation de lentille (avant/arrière), capture sur Canvas HTML5, conversion en fichier Blob/JPEG et prévisualisation instantanée.
Moteur de Formulaires Dynamiques	app.blade.php (renderDynamicFormFields)	Générateur d'interface dynamique pour 16 types de champs (Texte, Nombre, Date, Oui/Non, Radio, Checkbox, Photo WebRTC, Vidéo, QR code scan, GPS géolocalisé, Signature tactile Canvas, Document PDF).

3. FLUX TECHNIQUE DE LIAISON FRONTEND -> BACKEND

La communication entre l'application Single Page Application (SPA) mobile et le serveur Laravel s'effectue via un protocole RESTful sécurisé par jetons Bearer. Voici le cycle d'exécution complet d'une mission terrain :

- 1. Authentification & Initialisation** : L'utilisateur saisit ses identifiants dans la vue Login. La requête `POST /api/login` valide les crédeniels via `Auth::attempt()`, génère un jeton Sanctum (`createToken`), et le retourne au client. Le token est persisté localement (`localStorage.setItem('auth_token')`) et injecté dans le header `Authorization: Bearer` pour toutes les requêtes ultérieures.
- 2. Chargement du Dashboard & Liste des Missions** : La vue Dashboard effectue l'appel `GET /api/interventions` et `GET /api/notifications`. Les données reçues alimentent dynamiquement les cartes de KPIs (À réaliser, En cours, Terminées, Notifications non lues) et le conteneur des missions récentes.
- 3. Acceptation de l'Intervention** : Le technicien clique sur "Accepter la mission". Une requête `POST /api/interventions/{id}/accept` est transmise au serveur. Le service `InterventionService` fait passer le statut à `Acceptee` et notifie les administrateurs.
- 4. Démarrage & Suivi GPS** : Lors du démarrage (`POST /api/interventions/{id}/start`), l'application contrôle le mode de présence (`GPS`, `QR`, `NFC`, `Manuel`). Si le mode GPS est actif, une session de tracking est initialisée (`POST /api/interventions/{id}/tracking/start`) et une bannière de suivi active s'affiche. Un timer JavaScript transmet périodiquement la position GPS du technicien via `POST /tracking/sessions/{session}/points`.
- 5. Saisie du Formulaire Dynamique** : Le technicien accède à l'écran Formulaire. L'application récupère la structure du formulaire via `GET /api/interventions/{id}/formulaire`. La fonction JS `renderDynamicFormFields()` construit dynamiquement l'interface selon le type d'intervention (16 types de champs pris en charge). Lors de la soumission (`POST /api/interventions/{id}/formulaire`), le backend enregistre les réponses dans la table `reponses` et fait évoluer le statut vers `Formulaire rempli`.
- 6. Rédaction du Rapport & Capture Médias (WebRTC & QR)** : Dans la vue Rapport, le technicien renseigne les travaux effectués, les observations et les recommandations.
 - *Scan QR Code* : Le bouton "Scan" active le modal QR (`#modal-qr-scan`) exploitant le flux vidéo de la caméra pour lire et remplir le matricule de l'équipement (`qrcode_scanner`).
 - *Caméra Live WebRTC* : Le bouton "Caméra Live" active le modal `#modal-live-camera`. Un cliché haute définition est capturé sur un `HTML5` et converti en fichier Blob/JPEG (`captureLiveCameraPhoto()`).
 - *Soumission Multipart* : La méthode `handleRapportSubmit()` assemble les données textes, les fichiers photos (`photos[]`) et vidéos (`videos[]`) dans une structure `FormData` et l'expédie via `POST /api/interventions/{id}/rapport` (`content-type: multipart/form-data`).
- 7. Validation et Clôture** : Une fois le rapport soumis et l'intervention validée (`POST /api/interventions/{id}/validate`), le statut passe à `Terminee`. Le rapport peut alors être librement consulté et téléchargé en PDF (`GET /api/rapports/{id}/pdf`).